QUAD

Qualcomm Unified Agent for Developers

Engineering Design Document

Pavan R

Version	1.0
Date	April 2026
Status	Draft — For Engineering Review
Classification	Qualcomm Confidential — Internal Use Only
Stack	Python 3.11 · FastMCP · Pydantic v2 · Jinja2
Architecture	Mock-First · Adapter Pattern · 4-Layer MCP

Table of Contents

1. Executive Summary

QUAD (Qualcomm Unified Agent for Developers) is a Claude Code-powered AI agent that abstracts Qualcomm's entire SDK ecosystem — QNN, SNPE, Hexagon, Adreno, AIMET — behind a single conversational interface using the Model Context Protocol (MCP).

The system exposes five MCP tools that handle the complete AI inference workflow: hardware detection, model conversion, workload profiling, compute orchestration, and code generation — across three hardware platforms.

1.1 Key Design Decisions

Decision	Choice	Rationale
Language	Python 3.11	Native Qualcomm SDK bindings, ML ecosystem alignment
MCP Framework	FastMCP	Python-native, async, simple tool registration
Data Validation	Pydantic v2	Runtime type safety, automatic JSON schema generation
Code Generation	Jinja2	Industry-standard templating, multi-language support
Logging	structlog	Structured JSON output, context binding, zero-config
Testing	pytest + pytest-asyncio	Async support, fixtures, 85% coverage target
Architecture	Mock-First	Full development without hardware; config toggle for real SDKs
SDK Integration	Adapter Pattern	Isolate SDK specifics; swap/extend without tool changes

1.2 Platform Coverage

Platform	Chipset	NPU	Primary SDK	Priority
AI PC (Windows)	Snapdragon X Elite	45 TOPS	QNN / QAIRT	P1
Arduino UNO Q (Linux)	QCS2210 (RB1)	~1 TOPS (DSP)	SNPE	P2
Mobile (Android)	Snapdragon 8 Elite	48 TOPS	SNPE / QNN	P3

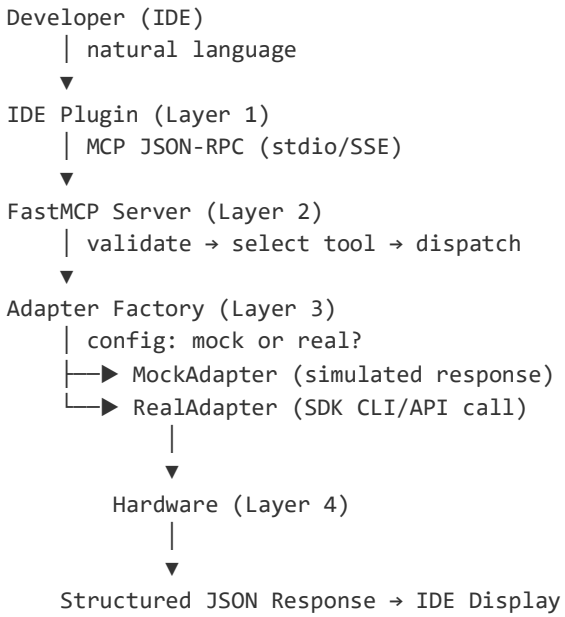
2. System Architecture

2.1 Four-Layer Architecture

Layer	Components	Responsibility
1 — Developer Interface	VS Code Extension · Arduino IDE Plugin · Android Studio Plugin	Natural language input, tool invocation UI, result visualization
2 — MCP Agent Server	FastMCP server · 5 tool handlers · Config manager · Logger	Tool dispatch, input validation, response formatting, error handling
3 — SDK Abstraction	QNN · SNPE · Hexagon · Adreno · AIMET · AI Hub · Mock adapters	SDK operations behind unified interface; mock/real switching via config
4 — Hardware Execution	CPU (Oryon/Kryo) · GPU (Adreno) · NPU (Hexagon HTP/DSP)	Physical compute dispatch, runtime management, power monitoring

2.2 Communication Flow

All communication follows a linear request-response pattern through the layers:



2.3 Mock-First Design Philosophy

Every adapter implements an identical abstract interface. The MockAdapter returns realistic, pre-computed responses derived from model metadata (file size, layer count, op types). This enables:

- **Full development and testing** without hardware or SDK installation
- **CI/CD pipelines** run on any machine (no Qualcomm hardware required)
- **Single config toggle** (adapter_mode = "mock" | "real") switches between backends

- **Graceful degradation** — if real SDK fails, fall back to mock with warning

3. Module Design

3.1 MCP Server Core — src/quad/server/

The server module handles FastMCP lifecycle, tool registration, and configuration.

Entry Point: main.py

```
from fastmcp import FastMCP
from quad.config import load_config
from quad.tools import register_all_tools
from quad.logging import setup_logging

mcp = FastMCP("QUAD — Qualcomm Unified Agent for Developers")
config = load_config()      # quad.toml + env overrides
setup_logging(config)       # structlog with JSON/console
register_all_tools(mcp, config)
mcp.run()                   # stdio or SSE transport
```

3.2 MCP Tool Implementations — src/quad/tools/

3.2.1 hardware_detect

Detects Qualcomm chipset and available compute unit topology.

Attribute	Specification
Input	platform: Literal["windows", "linux", "android"]
Output	DeviceProfile: chipset, cpu_cores, cpu_arch, cpu_freq_ghz, gpu_model, gpu_tflops, npu_model, npu_tops, ram_gb, sdk_path, sdk_version, available_runtimes
Mock	Returns pre-defined profile from configs/device_profiles/{platform}.json
Real	Windows: WMI + QNN device query · Linux: /proc/cpuinfo + SNPE · Android: ADB getprop
Errors	PlatformNotDetectedError · SDKNotFoundError
Timeout	30s (result cached for session duration)
Caching	Yes — hardware does not change during session

3.2.2 convert_model

Converts ML models from source framework to QNN/SNPE format with optional quantization.

Attribute	Specification
Input	source_format: Literal["onnx","pytorch","tensorflow","tflite"] · model_path: str · target_sdk: Literal["qnn","snpe"] · quantization: Literal["fp32","int8","int4"]
Output	ConversionResult: output_path, model_size_mb, original_size_mb, supported_ops_pct,

	unsupported_ops[], quantization_applied, conversion_time_s
Mock	Parse ONNX metadata → simulate conversion time → estimate size reduction per quantization level
Real (QNN)	qnn-onnx-converter → qnn-model-lib-generator → qnn-context-binary-generator
Real (SNPE)	snpe-onnx-to-dlc → snpe-dlc-quantize (if INT8/INT4)
Errors	ModelNotFoundError · UnsupportedFormatError · ConversionFailedError · QuantizationError
Timeout	300s (large model conversion)
Validation	Output file exists, size > 0; smoke-test with dummy input

3.2.3 profile_workload

Invokes platform-appropriate profiler and returns structured performance/power metrics.

Attribute	Specification
Input	model_path: str · platform: str · runtime: Literal["cpu","gpu","npu","auto"] · duration_s: int = 10 · iterations: int = 100
Output	ProfilingReport: latency{mean,p50,p95,p99,min,max}, throughput_fps, power_mw, memory_peak_mb, utilization{cpu,gpu,npu}, layers[]
Mock	Metrics from model size: latency ≈ size_mb × 0.5ms (NPU), × 2.0ms (CPU)
Real	Windows: QPM3 + Snapdragon Profiler · Linux: snpe-throughput-net-run · Android: Perfetto
Errors	ProfilerNotFoundError · DeviceNotConnectedError · ProfilingTimeoutError
Timeout	120s + duration_s

3.2.4 orchestrate_workload

Allocates inference graph nodes across CPU/GPU/NPU based on profiling data and power mode.

Attribute	Specification
Input	model_path: str · profile_report: ProfilingReport · power_mode: Literal["performance","balanced","efficiency"]
Output	AllocationMap: allocation{layer→runtime}, projected_latency_ms, projected_power_mw, fallback_layers[]
Algorithm	performance: all compatible → NPU · balanced: top 70% → NPU, 20% → GPU · efficiency: minimize power, CPU-first
Constraints	Memory: sum(layers) < device_ram × 0.8 · Power: per-mode budget · Ops: NPU compatibility check
Errors	InvalidProfileError · MemoryExceededError
Timeout	60s

3.2.5 generate_code

Generates platform-specific inference code using Jinja2 templates.

Attribute	Specification
Input	platform: str · sdk: Literal["qnn","snpe"] · language: Literal["cpp","python","java","kotlin","arduino_sketch"] · model_path: str · allocation_map: AllocationMap (optional)
Output	GeneratedCode: source_files{filename→content}, build_instructions, dependencies[], sample_input, expected_output_format
Templates	templates/{platform}/{language}/inference.{ext}.j2
Validation	Python: ast.parse() · C++: bracket balance · Kotlin/Java: basic syntax · All: no unrendered {{ }}
Errors	UnsupportedLanguageError · TemplateRenderError · SDKPathNotConfiguredError
Timeout	30s

3.3 SDK Adapter Layer — src/quad/adapters/

Abstract Base Class

```
class SDKAdapter(ABC):
    """Base interface for all Qualcomm SDK adapters."""

    @abstractmethod
    async def detect_hardware(self, platform: str) -> DeviceProfile: ...

    @abstractmethod
    async def convert_model(self, request: ConversionRequest) -> ConversionResult: ...

    @abstractmethod
    async def profile(self, request: ProfileRequest) -> ProfilingReport: ...

    @abstractmethod
    async def get_supported_ops(self) -> list[str]: ...

    @abstractmethod
    async def execute_inference(self, model_path: str, input_data: Any) -> Any: ...
```

Adapter Registry

Adapter	SDK	Key Operations
QNNAdapter	QNN SDK 2.x	qnn-onnx-converter · qnn-model-lib-generator · qnn-net-run · qnn-context-binary-generator
SNPEAdapter	SNPE SDK 2.x	snpe-onnx-to-dlc · snpe-dlc-quantize · snpe-net-run · snpe-throughput-net-run
HexagonAdapter	Hexagon SDK 5.x	hexagon-clang · hexagon-sim ·

		custom op compilation
AdrenoAdapter	Adreno SDK	OpenCL kernel dispatch · Vulkan compute pipeline
AIMETAdapter	AIMET	QuantizationSimModel · CrossLayerEqualization · Adaround
AIHubAdapter	AI Hub API	REST: /v1/models/profile · /v1/models/compile · /v1/devices
MockAdapter	(none)	Returns pre-computed realistic responses for all operations

Adapter Factory

```
class AdapterFactory:
    """Config-driven adapter selection."""

    def __init__(self, config: ServerConfig):
        self._mode = config.adapter_mode  # "mock" or "real"
        self._registry: dict[str, type[SDKAdapter]] = {
            "qnn": QNNAdapter,
            "snpe": SNPEAdapter,
            "hexagon": HexagonAdapter,
            "adreno": AdrenoAdapter,
            "aimet": AIMETAdapter,
            "aihub": AIHubAdapter,
        }

    def get_adapter(self, sdk: str) -> SDKAdapter:
        if self._mode == "mock":
            return MockAdapter(sdk_name=sdk)
        return self._registry[sdk](self._config)
```

3.4 Platform Layer — src/quad/platforms/

Platform	Detection	Communication	Key Operations
WindowsPlatform	WMI (Win32_Processor, Win32_VideoController)	Local subprocess	QNN SDK invocation, QPM3 launch, file I/O
LinuxPlatform	/proc/cpuinfo + /sys/ + SSH remote	SSH (asyncssh) + SCP	Remote deployment, SNPE execution, serial debug
AndroidPlatform	ADB getprop, dumpsys	ADB shell/push/pull	Library push, APK deploy, logcat, thermal monitoring

3.5 Code Generation Engine — src/quad/codegen/

Jinja2-based multi-language code generation. Templates produce complete, compilable projects.

Language	Output Files	Build System	Platform

C++ (QNN)	main.cpp · inference.h · CMakeLists.txt	CMake 3.22+	Windows
Python (QNN)	inference.py · requirements.txt	pip	Windows
Python (SNPE)	inference.py · requirements.txt	pip	Linux
Arduino Sketch	inference.ino · inference.h · CMakeLists.txt	Arduino CLI / CMake	Linux
Kotlin (Android)	InferenceEngine.kt · build.gradle.kts · jni/inference_jni.cpp	Gradle + CMake (NDK)	Android

4. Data Models (Pydantic Schemas)

All data contracts are defined as Pydantic v2 BaseModel classes in src/quad/models/. These provide runtime validation, automatic JSON Schema generation, and IDE autocompletion.

4.1 DeviceProfile

```
class DeviceProfile(BaseModel):
    chipset: str              # "Snapdragon X Elite X1E-80-100"
    platform: Literal["windows", "linux", "android"]
    cpu_cores: int
    cpu_arch: str             # "Oryon ARM64"
    cpu_freq_ghz: float
    gpu_model: str            # "Adreno X1-85"
    gpu_tflops: float
    npu_model: str            # "Hexagon NPU"
    npu_tops: float
    ram_gb: float
    sdk_path: str | None
    sdk_version: str | None
    available_runtimes: list[Literal["cpu", "gpu", "npu"]]
```

4.2 ConversionResult

```
class ConversionResult(BaseModel):
    output_path: str
    model_size_mb: float
    original_size_mb: float
    compression_ratio: float
    supported_ops_pct: float       # 0-100
    unsupported_ops: list[str]
    quantization_applied: str      # "fp32", "int8", "int4"
    conversion_time_s: float
    target_sdk: str
    warnings: list[str]
```

4.3 ProfilingReport

```
class LatencyStats(BaseModel):
    mean_ms: float
    p50_ms: float
    p95_ms: float
    p99_ms: float
    min_ms: float
    max_ms: float

class LayerProfile(BaseModel):
    name: str
    op_type: str
```

```
    runtime: Literal["cpu", "gpu", "npu"]
    latency_ms: float
    memory_mb: float

class ProfilingReport(BaseModel):
    latency: LatencyStats
    throughput_fps: float
    power_mw: float
    memory_peak_mb: float
    memory_avg_mb: float
    utilization: dict[str, float]  # {"cpu": 15.2, "gpu": 5.0, "npu": 89.3}
    layers: list[LayerProfile]
    device: DeviceProfile
    runtime_used: str
    duration_s: float
```

4.4 AllocationMap

```
class AllocationMap(BaseModel):
    allocation: dict[str, Literal["cpu", "gpu", "npu"]]
    projected_latency_ms: float
    projected_power_mw: float
    projected_memory_mb: float
    power_mode: Literal["performance", "balanced", "efficiency"]
    fallback_layers: list[str]
    npu_utilization_pct: float
    gpu_utilization_pct: float
    cpu_utilization_pct: float
```

4.5 GeneratedCode

```
class GeneratedCode(BaseModel):
    source_files: dict[str, str]   # filename -> content
    build_instructions: str
    dependencies: list[str]
    language: str
    platform: str
    sdk: str
    sample_input: str
    expected_output_format: str
```

4.6 ServerConfig

```
class ServerConfig(BaseSettings):
    adapter_mode: Literal["mock", "real"] = "mock"
    log_level: Literal["debug","info","warning","error"] = "info"
    log_format: Literal["json", "console"] = "console"
    qnn_sdk_path: str | None = None
```

```
snpe_sdk_path: str | None = None
hexagon_sdk_path: str | None = None
adreno_sdk_path: str | None = None
ai_hub_api_key: str | None = None
cache_hardware_detection: bool = True
default_platform: str | None = None
model_output_dir: str = "./output"
template_dir: str = "./templates"
```

5. API Contracts & Error Codes

5.1 MCP Tool Registration

Each tool is registered via @mcp.tool() decorator with JSON Schema input validation (auto-generated from Pydantic models).

5.2 Error Codes

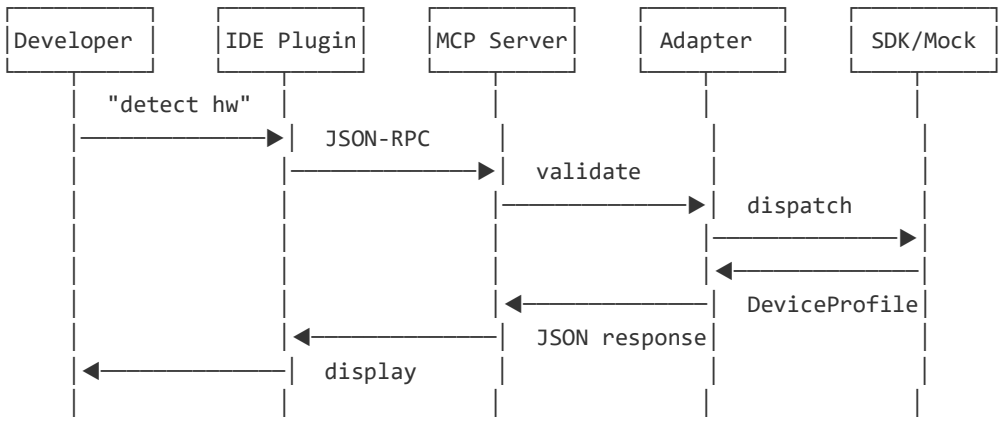
Code	HTTP Equiv	Description	Recoverable
PLATFORM_NOT_DETECTED	404	Target platform/device not found	No
SDK_NOT_FOUND	404	Required SDK not installed at configured path	No
SDK_VERSION_MISMATCH	409	SDK version incompatible with operation	No
MODEL_NOT_FOUND	404	Model file does not exist at specified path	No
UNSUPPORTED_FORMAT	400	Model format not supported for conversion	No
CONVERSION_FAILED	500	Model conversion process failed	Maybe
PROFILER_TIMEOUT	408	Profiling exceeded timeout duration	Yes (retry)
DEVICE_NOT_CONNECTED	503	Target device not reachable	Yes (reconnect)
MEMORY_EXCEEDED	507	Operation exceeds device memory limits	No
TEMPLATE_ERROR	500	Code generation template rendering failed	No

5.3 Timeout Policies

Tool	Default Timeout	Notes
hardware_detect	30s	Cached after first call per session
convert_model	300s	Large models (>500MB) may need longer
profile_workload	120s + duration_s	Profiler capture + analysis time
orchestrate_workload	60s	Algorithm computation
generate_code	30s	Template rendering + validation

6. Sequence Diagrams

6.1 End-to-End Workflow



6.2 Model Conversion Pipeline

```
1. convert_model(source_format="onnx", model_path, target_sdk="qnn", quantization="int8")
2. Validate: model_path exists? format matches extension?
3. Select adapter: target_sdk="qnn" → QNNAdapter (or MockAdapter)
4. Adapter pipeline:
   ├── qnn-onnx-converter --input model.onnx --output model_qnn/
   ├── qnn-model-lib-generator --model model_qnn/ --output lib/
   └── qnn-context-binary-generator --model lib/ --output model.bin
5. Parse stdout/stderr → extract unsupported_ops[], warnings[]
6. Validate: output file exists? size > 0?
7. Optional: smoke-test with qnn-net-run --input random_tensor
8. Return ConversionResult{output_path, size, ops_pct, ...}
```

6.3 Orchestration Algorithm

```
1. Input: model layers[] + profile_report + power_mode
2. For each layer:
   └── Check: op_type in npu_supported_ops?
       ├── YES → candidate for NPU
       └── NO  → forced to CPU (added to fallback_layers)
3. Apply power_mode strategy:
   ├── "performance" → all NPU-compatible → NPU
   ├── "balanced"    → top 70% compute → NPU, 20% → GPU, 10% → CPU
   └── "efficiency"  → only highest-compute → NPU, rest → CPU
4. Memory check: sum(allocated_memory) < device_ram × 0.8?
   └── If exceeded: move lowest-priority layers NPU→GPU→CPU until within budget
5. Calculate projections from profile data × runtime speedup factors
6. Return AllocationMap
```

7. Error Handling Strategy

7.1 Exception Hierarchy

```
QUADError (base)
├── PlatformError
│   ├── PlatformNotDetectedError
│   └── DeviceNotConnectedError
├── SDKError
│   ├── SDKNotFoundError
│   ├── SDKVersionMismatchError
│   └── SDKExecutionError
├── ConversionError
│   ├── UnsupportedFormatError
│   ├── ConversionFailedError
│   └── QuantizationError
├── ProfilingError
│   ├── ProfilerNotFoundError
│   ├── ProfilingTimeoutError
│   └── InsufficientMemoryError
├── OrchestrationError
│   ├── InvalidProfileError
│   └── MemoryExceededError
└── CodegenError
    ├── TemplateRenderError
    └── UnsupportedLanguageError
```

7.2 Graceful Degradation

Failure	Degradation Path	User Impact
NPU unavailable	NPU → GPU → CPU fallback	Higher latency, same correctness
Real SDK unavailable	Switch to mock mode with warning	Simulated metrics (clearly labeled)
Profiler unavailable	Return estimated metrics from model metadata	Less accurate, flagged as estimate
Device disconnected	Cache last-known profile, warn about staleness	Stale data, clearly labeled
Template error	Return partial code with error annotation	Incomplete but debuggable output

7.3 Retry Policy

Error Class	Retry?	Strategy
Transient (timeout, connection reset)	Yes	3 attempts · exponential backoff (1s, 2s, 4s)
SDK execution failure	Once	Retry with --verbose for diagnostics
Permanent (not found, unsupported)	No	Return error immediately with fix suggestion

8. Configuration System

QUAD uses TOML configuration with environment variable overrides (QUAD_* prefix).

8.1 Configuration File: quad.toml

```
[server]
adapter_mode = "mock"           # "mock" or "real"
log_level = "info"              # debug, info, warning, error
log_format = "console"          # "json" for production
model_output_dir = "./output"
template_dir = "./templates"

[adapters.qnn]
sdk_path = "C:/Qualcomm/AIStack/QNN"
version = "2.x"
target_arch = "aarch64-windows"

[adapters.snpe]
sdk_path = "/opt/snpe-2.x"
version = "2.x"
target_arch = "aarch64-linux"

[adapters.hexagon]
sdk_path = "/opt/hexagon-sdk"
tools_path = "/opt/hexagon-tools"

[adapters.ai_hub]
api_key_env = "QAI_HUB_API_KEY"    # reads from env var
base_url = "https://app.aihub.qualcomm.com/api/v1"

[platforms.windows]
enabled = true
device_type = "local"

[platforms.linux]
enabled = true
device_type = "remote"
ssh_host = "arduino-uno-q.local"
ssh_user = "root"
ssh_key = "~/.ssh/id_arduino"

[platforms.android]
enabled = true
device_serial = ""               # auto-detect if empty
adb_path = "adb"
```

8.2 Environment Variable Overrides

All config values can be overridden with QUAD_ prefix: QUAD_ADAPTER_MODE=real · QUAD_QNN_SDK_PATH=/custom/path · QUAD_LOG_LEVEL=debug

9. Testing Strategy

9.1 Test Pyramid

Level	Count	Speed	Coverage	What It Tests
Unit	~200	< 30s	85% lines	Tool logic, adapter methods, model validation
Integration	~50	< 2 min	All happy + error paths	MCP request → response cycle
Protocol	~20	< 30s	JSON-RPC spec	MCP protocol conformance
E2E	~10	< 5 min	Full workflows	detect → convert → profile → orchestrate → generate

9.2 Test Organization

```
tests/
├── conftest.py                # Shared fixtures, mock adapter, sample configs
├── unit/
│   ├── test_tools/            # One file per tool
│   ├── test_adapters/         # One file per adapter
│   ├── test_platforms/        # Platform detection logic
│   ├── test_codegen/          # Template rendering
│   └── test_models/           # Pydantic validation (valid + invalid)
├── integration/
│   ├── test_mcp_protocol.py   # Full JSON-RPC cycle
│   └── test_workflows.py      # Multi-tool sequences
├── e2e/
│   ├── test_windows_e2e.py
│   ├── test_linux_e2e.py
│   └── test_android_e2e.py
└── fixtures/
    ├── models/                 # Sample .onnx files
    ├── configs/                # Test quad.toml variants
    └── expected_outputs/       # Golden files for comparison
```

10. Security Considerations

Threat	Mitigation	Implementation
Credential leakage	API keys in env vars only; stripped from logs	structlog processor filters key patterns
Path traversal	Validate all paths against allowed directories	Reject ../ sequences; resolve to absolute path
Command injection	subprocess with arg list, never shell=True	All SDK CLI calls use parameterized invocation
ADB injection	Validate device serial; sanitize shell args	Allowlist device serials from adb devices
Malicious models	Check file headers/magic bytes; size limits	Reject files > configured max_model_size_gb
Log poisoning	Sanitize user-provided strings in log output	structlog processors escape control characters
Network exposure	HTTPS only; cert validation; no proxy cred logging	httpx with verify=True by default

11. IDE Plugin Specifications

11.1 VS Code Extension

Attribute	Value
Language	TypeScript
Activation	Workspace contains quad.toml OR manual command
MCP Transport	stdio (spawn quad-server as child process)
Commands	QUAD: Detect Hardware · Convert Model · Profile · Orchestrate · Generate Code
UI	Output panel (JSON) · Status bar (connected/disconnected) · Webview (profiling charts)
Settings	quad.serverCommand · quad.configPath · quad.adapterMode
Distribution	VS Code Marketplace (.vsix)

11.2 Arduino IDE Plugin

Attribute	Value
Language	Java (Arduino IDE 2.x Eclipse-based)
Activation	Board selection: "Arduino UNO Q (QCS2210)"
MCP Transport	HTTP/SSE to local FastMCP server
Commands	Tools menu: Deploy AI Model · Profile Inference · Generate Sketch
Integration	Board manager (custom board definition) · Serial monitor output parsing
Distribution	Arduino Library Manager

11.3 Android Studio Plugin

Attribute	Value
Language	Kotlin (IntelliJ Platform SDK)
Activation	Project contains SNPE/QNN dependencies in build.gradle
MCP Transport	stdio to local quad-server
Commands	Tool window: Device Selector · Model Converter · Profiler · Code Generator
Integration	ADB panel · Run configuration: "QUAD AI Inference"
Distribution	JetBrains Marketplace

12. Deployment & Packaging

Artifact	Format	Distribution	Command
QUAD Server	Python wheel	PyPI (or internal registry)	pip install quad-agent
Docker Image	OCI container	GitHub Container Registry	docker pull ghcr.io/quad-agent:latest
VS Code Extension	.vsix	VS Code Marketplace	code --install-extension quad.vsix
Arduino Plugin	.jar	Arduino Library Manager	Board Manager → Install
Android Studio Plugin	.jar/.zip	JetBrains Marketplace	Settings → Plugins → Install

12.1 Claude Code Integration

```
# .claude/settings.json
{
  "mcpServers": {
    "quad": {
      "command": "quad-server",
      "args": ["--config", "quad.toml"],
      "env": {"QAI_HUB_API_KEY": "${QAI_HUB_API_KEY}"}
    }
  }
}
```

13. Cursor & Tool Plugin Configuration

13.1 Cursor Rules (.cursor/rules/quad.mdc)

```
---
description: QUAD project development rules
globs: ["src/**/*.py", "tests/**/*.py"]
---

## Architecture Rules
- All SDK interactions go through adapters in src/quad/adapters/
- Never import SDK modules directly in tool handlers
- All tool inputs/outputs use Pydantic models from src/quad/models/
- MockAdapter must be updated when any adapter interface changes

## Code Style
- async/await for all I/O · Type hints on all public functions
- structlog for logging · QUADError subclasses for errors

## Testing
- Every method needs a unit test · Mock mode works without SDK
- Use pytest fixtures from conftest.py
```

13.2 GitHub Copilot (.github/copilot-instructions.md)

Provides Copilot with project context: architecture patterns, directory layout, coding standards, and anti-patterns to avoid.

13.3 Pre-commit Hooks (.pre-commit-config.yaml)

```
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    hooks: [ruff, ruff-format]
  - repo: https://github.com/pre-commit/mirrors-mypy
    hooks: [mypy]
```

13.4 VS Code Workspace (.vscode/settings.json)

Configures Python interpreter, pytest discovery, ruff formatter, and file exclusions for the development environment.

14. Appendix: Complete File Map

```
QUAD/
├── CLAUDE.md                          # Claude Code context & session resume
├── README.md                          # Project overview & quick start
├── pyproject.toml                     # Dependencies & project metadata
├── quad.toml                          # Runtime configuration
├── Makefile                           # Dev commands (serve, test, lint)
├── Dockerfile                         # CI/CD container image
├── .pre-commit-config.yaml           # Code quality hooks
├── .cursor/rules/quad.mdc            # Cursor IDE rules
├── .github/
│   ├── copilot-instructions.md       # Copilot context
│   └── workflows/ci.yml             # GitHub Actions CI
├── .vscode/settings.json             # VS Code workspace
├── .claude/settings.json             # MCP server registration
├── docs/                             # Documentation
├── src/quad/
│   ├── server/main.py                # FastMCP entry point
│   ├── tools/                        # 5 MCP tool handlers
│   │   ├── hardware_detect.py
│   │   ├── convert_model.py
│   │   ├── profile_workload.py
│   │   ├── orchestrate_workload.py
│   │   └── generate_code.py
│   ├── adapters/                     # SDK abstraction layer
│   │   ├── base.py                   # SDKAdapter ABC
│   │   ├── factory.py                # AdapterFactory
│   │   ├── mock_adapter.py           # Mock (no SDK needed)
│   │   ├── qnn_adapter.py
│   │   ├── snpe_adapter.py
│   │   ├── hexagon_adapter.py
│   │   ├── adreno_adapter.py
│   │   ├── aimet_adapter.py
│   │   └── aihub_adapter.py
│   ├── platforms/                    # Platform-specific logic
│   │   ├── base.py
│   │   ├── windows.py
│   │   ├── linux.py
│   │   └── android.py
│   ├── codegen/                      # Code generation engine
│   │   ├── engine.py                 # Jinja2 template renderer
│   │   └── validators.py             # Output syntax validation
│   ├── models/                       # Pydantic data models
│   │   ├── device.py
│   │   ├── conversion.py
│   │   ├── profiling.py
│   │   ├── orchestration.py
│   │   ├── codegen.py
│   │   ├── config.py
│   │   └── errors.py
│   ├── config.py                     # TOML config loader
```

```
│   ├── exceptions.py              # Exception hierarchy
│   └── logging.py                 # Structlog setup
├── templates/                     # Jinja2 code templates
│   ├── windows/{cpp,python}/
│   ├── linux/{arduino_sketch,python}/
│   └── android/{kotlin,jni}/
├── tests/                         # Test suites
│   ├── unit/ · integration/ · e2e/
│   └── fixtures/
├── plugins/                       # IDE extensions
│   ├── vscode/                    # TypeScript
│   ├── arduino/                   # Java
│   └── android-studio/            # Kotlin
└── configs/
    ├── quad.toml.example
    ├── device_profiles/           # Mock device data
    └── supported_ops/             # Op compatibility lists
```

Module Dependency Order (Build Bottom-Up)

Order	Module	Depends On
1	src/quad/models/	None (pure data definitions)
2	src/quad/exceptions.py	None
3	src/quad/config.py	models/config.py
4	src/quad/logging.py	config
5	src/quad/adapters/base.py	models
6	src/quad/adapters/mock_adapter.py	base, models
7	src/quad/adapters/factory.py	all adapters
8	src/quad/platforms/	models, exceptions
9	src/quad/codegen/	models
10	src/quad/tools/	adapters, platforms, codegen, models
11	src/quad/server/main.py	tools, config, logging